ANSHIKA SINGH.

23BCS11326.

23KRG-2A.

PRATICAL EST - CC.

Q1 Travelling Salesman (TSP).

Approach →

1) Start from city 0.
2.) Visit all the cities once.
3.) Return Back to city 0.
4.) Minimize total Cost.

Psuedo code :-

```
int solve (int mask, int pos) {
  if (mask == (1<<n) - 1) {
    return cost [pos][0];
  }
  if( dp[mask][pos] != -1) return dp[mask][pos];

  int ans = 1e9;

  for( int i = 0; i<n; i++){
    if ((mask & (1 << i))==0) {
      int ans2 = cost[pos][i] + solve(mask |
      (1 << i), i);
      ans = min( ans, ans2);
    }
  }
  return dp[mask][pos] = ans;
```

→ **Time Complexity:** $O(2^n \times n^2)$.

2^n states (mask).

n → cities.

→ **Space Complexity:** $O(n \times 2^n)$.

→ **Eg →**

2			
0	111.		
112	0.		

output.

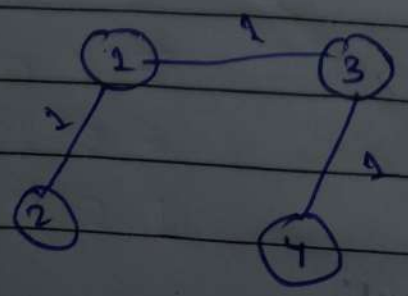
223.

Q2 Prim's Algorithm (MST).

→ **Approach:**

→ Connect all vertacies.
→ With minimum total weight.
→ Without forming a cycle.

as MST has → n nodes, n-1 edges.



→ Start from any node (usually we take 0).
→ Always we have to pick minimum weight edge that connects a new node.

→ Steps:

1.) Create a adj list from edges.
2.) Use Min Heap priority queue.
3.) Visited Array
4.) Start from node 0.
5.) Pick the smallest edge → Add to MST.
6.) Repeat.

Pseudo code:

```
vector<pair<int, int>> adj [1000];
for (int i=0; i<E; i++){
    int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
priority_queue<pair<int, int>, vector<pair<int, int>>,
        greater<pair<int, int>>> pq;

Vector<bool> vis(V, false);
pq.push({0, 0});
int sum=0;
```

```
while (!pq.empty()){
  int wt = pq.top().first;
  int node = pq.top().second;
  pq.pop();
  if (vis[node] == true) continue;
  vis[node] = true;
  sum = sum + wt;

  for (auto it : adj[node]){
    int nextn = it.first;
    int edgen = it.second;
    if (!vis[nextn]){
      pq.push({edgewt, nextn});
    }
  }
}
```

→ Time complexity : $O(E \log V)$.

→ Space complexity : $O(V+E)$.

→ Eg :

			output
3	3		
0	1	5	4
1	2	3	
0	2	1	